



WebGL e Funzioni Utili

m4.js
webgl-utils.js
dat.gui.js
glm_utils.js
mesh_utils.js

near		1
far		100
D		5
theta		0.78
phi		0.78
fovy		40
rotationAxis	0 ▾	
rotation	☐	
Close Controls		



Libreria m4.js

Vediamo di usare una libreria di funzioni per definire le matrici V e P e in genere per gestire operazioni fra vettori e matrici.

$$V = B^{-1} \quad \text{con} \quad B = \begin{pmatrix} \boxed{\underline{X_e}} & \boxed{\underline{Y_e}} & \boxed{\underline{Z_e}} & \begin{matrix} D \sin\varphi \cos\theta \\ D \sin\varphi \sin\theta \\ D \cos\varphi \\ 1 \end{matrix} \\ 0 & 0 & 0 & \end{pmatrix}$$

$$P = \begin{pmatrix} f/ar & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



Libreria m4.js

Per definire la matrice di vista V solitamente si fa uso di una funzione chiamata **lookAt** con i seguenti parametri:

```
function lookAt( eye, at, up ){  
...  
}
```

eye: camera position
at : target position
up : View-Up vector

```
// Compute the camera's matrix using look at.  
var cameraMatrix = m4.lookAt(cameraPos, cameraTar, up);  
// Make a view matrix from the camera matrix.  
var viewMatrix = m4.inverse(cameraMatrix);
```



Libreria m4.js

Per definire la matrice di proiezione P solitamente si fa uso di una funzione chiamata **perspective** con i seguenti parametri:

```
function perspective( fovy, aspect, near, far ){  
    ...  
}
```

fovy : field of view in y axis in radians
aspect : aspect ratio of viewport (width / height)
near :near Z clipping plane
far :far Z clipping plane

```
// Compute the projection matrix  
var projectionMatrix m4.perspective(fieldOfViewRadians,  
    aspect, zNear, zFar);
```



m4.js e Typed Array

La libreria **m4.js** restituisce per default array tipizzati e per la precisione **Float32Array**.

Gli array tipizzati sono stati progettati dal comitato degli standard WebGL, per motivi di prestazioni.

Gli array Javascript sono generici e possono contenere oggetti, altri array ecc., e gli elementi non sono necessariamente sequenziali in memoria, come invece sono nel linguaggio C.

WebGL richiede che i buffer siano sequenziali in memoria, perché è così che l'API C si aspetta che siano.



m4.js e Typed Array

Nota: alcune funzioni come `array.push` non funzionano bene su array tipizzati. E' per questo che si usa la libreria `m4.js` che lavora e restituisce, su richiesta, anche array non tipizzati.

```
var t1 = m4.subtractVectors(vertices[b], vertices[a]);
var t2 = m4.subtractVectors(vertices[c], vertices[b]);
//restituisce un typed array Float32Array
normal = m4.cross(t1, t2); //restituisce un array tipizzato
```

```
var t1 = m4.subtractVectors(vertices[b], vertices[a]);
var t2 = m4.subtractVectors(vertices[c], vertices[b]);
//restituisce un JavaScript array
var normal=[ ];
normal = m4.cross(t1, t2, normal); //restituisce un array
                                   //non tipizzato
```



m4.js functions

copy,
lookAt,
addVectors,
subtractVectors,
distance,
distanceSq,
normalize,
compose,
cross,
decompose,
mvec4,
dot,
identity,
transpose,
length,
orthographic,
frustum,

perspective,
translation,
translate,
xRotation,
yRotation,
zRotation,
xRotate,
yRotate,
zRotate,
axisRotation,
axisRotate,
scaling,
scale,
flatten,
multiply,
inverse,
transformVector,

transformPoint,
transformDirection,
transformNormal,
setDefaultType



Libreria webgl-utils.js

Useremo, per comodità anche altre semplici librerie, con il solo scopo di ridurre il numero di linee di codice da scrivere; per esempio useremo la libreria **webgl-utils.js** che mette a disposizione alcune funzioni utili per compilare, linkare e ottenere uno shader program.

```
function createProgramFromScripts(gl, shaderScriptIds, ... ) {  
...  
}
```

shaderScriptIds :Array of ids of the script tags for the shaders. The first is assumed to be the vertex shader, the second the fragment shader.

```
// setup GLSL program  
var program = WebGLUtils.createProgramFromScripts(gl,  
    ["3d-vertex-shader", "3d-fragment-shader"]);
```




Libreria webgl-utils.js

La libreria **webgl-utils.js** propone anche altre funzioni utili:

createAugmentedTypedArray,
createAttribsFromArrays,
createBuffersFromArrays,
createBufferInfoFromArrays,
createAttributeSetters,
createProgram,
createProgramFromScripts,
createProgramFromSources,
createProgramInfo,
createUniformSetters,
createVAOAndSetAttributes,
createVAOFromBufferInfo,
drawBufferInfo,
drawObjectList,
glEnumToString,

getExtensionWithKnownPrefixes,
resizeCanvasToDisplaySize,
setAttributes,
setBuffersAndAttributes,
setUniforms



Libreria dat.gui.js

Alcuni programmatori di Google hanno creato una libreria chiamata **dat.gui**, che permette di creare molto facilmente una semplice interfaccia utente che permette di modificare le variabili utilizzate in un codice Javascript.
Per la documentazione si veda:

<https://github.com/dataarts/dat.gui>

E' la libreria utilizzata da **three.js** per mostrare e debuggare gli esempi.



Esempio

Cartella HTML5_webgl_3

Vediamo un esempio in cui si visualizza un cubo colorato e si utilizzano le librerie `webgl-utils.js`, `m4.js` e `dat.gui.js`.

Nella cartella tutti gli esempi che usano la libreria `dat.gui.js` hanno un nome che finisce con: `_GUI.html` e `_GUI.js`

In questi codici i programmi shader vengono scritti come script definendo `type = "not-javascript"`.

Per tutte le operazioni necessarie per compilare e linkare i programmi shader si usa la libreria `webgl-utils.js`.



Esempio

`cube_color_shading_GUI.html` e `.js`

In questo codice l'oggetto cubo ruota sull'asse coordinato selezionato, mentre è possibile modificare la camera e la proiezione; i parametri della camera vengono definiti richiamando la function `m4.lookAt`, quelli di proiezione richiamando la function `m4.perspective`, entrambe della libreria `m4.js`

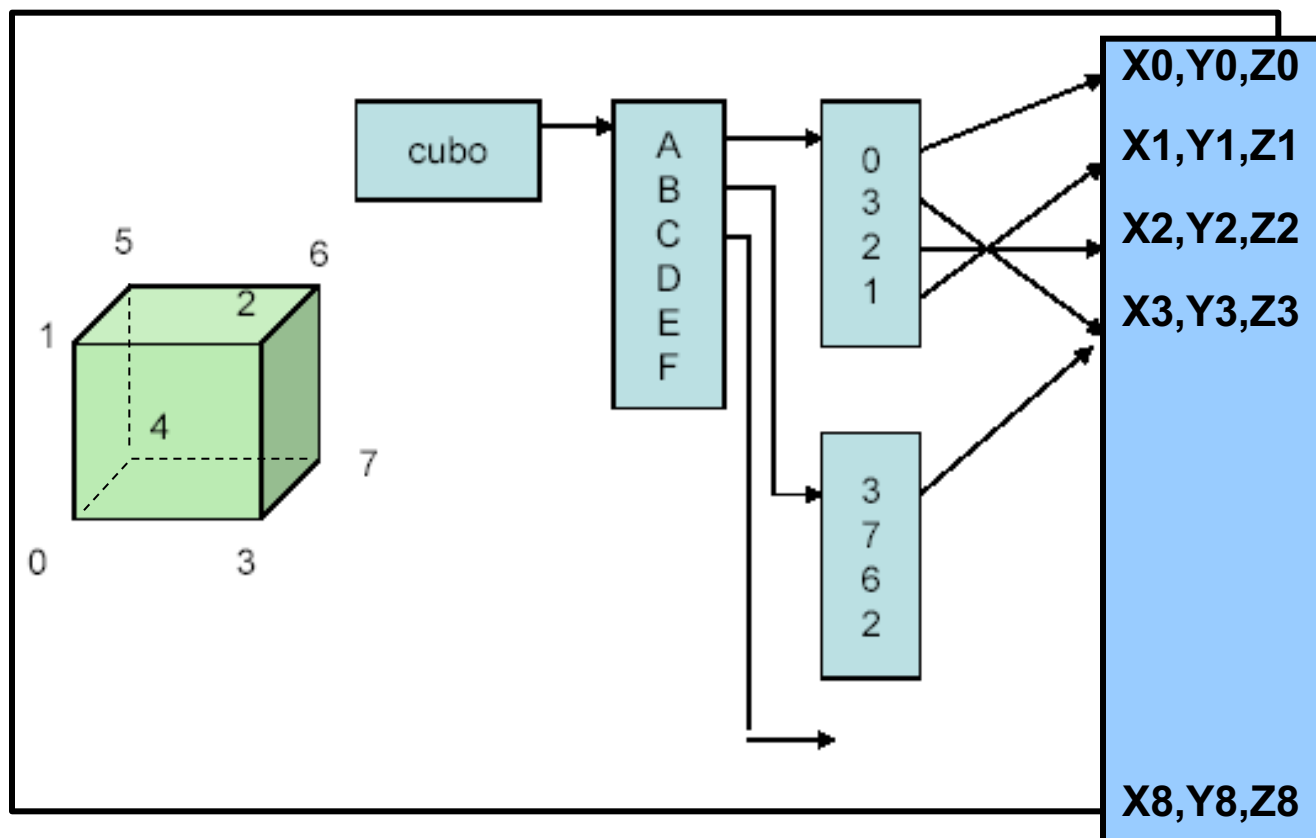
Il cubo viene rappresentato in modalità Gouraud shading grazie ad un modello di illuminazione di Phong applicato ai vertici.

In questo esempio viene definita una GUI utilizzando la libreria `dat.gui.js`

Definiamo un Cubo (1/5)

Vediamo come nel codice si definisce la geometria cubo e i colori per ogni faccia.

Oggetto mesh cubo (6 facce e 8 vertici) con un colore per ogni faccia:



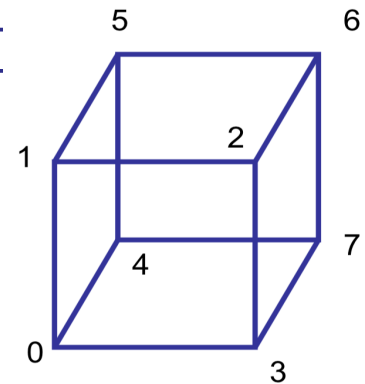
Definiamo un Cubo (2/5)

Definiamo gli 8 vertici del cubo in coordinate omogenee:

```
var vertices = [ [ 1.0, -1.0, -1.0, 1.0,], [ 1.0, -1.0, 1.0, 1.0,],  
                [ 1.0, 1.0, 1.0, 1.0,], [ 1.0, 1.0, -1.0, 1.0,],  
                [-1.0, -1.0, -1.0, 1.0,], [-1.0, -1.0, 1.0, 1.0,],  
                [-1.0, 1.0, 1.0, 1.0,], [-1.0, 1.0, -1.0, 1.0,] ]
```

e definiamo i colori per vertice:

```
var vertexColors = [ [0.0, 0.0, 0.0, 1.0,], // black  
                    [1.0, 0.0, 0.0, 1.0,], // red  
                    [1.0, 1.0, 0.0, 1.0,], // yellow  
                    [0.0, 1.0, 0.0, 1.0,], // green  
                    [0.0, 0.0, 1.0, 1.0,], // blue  
                    [1.0, 0.0, 1.0, 1.0,], // magenta  
                    [0.0, 1.0, 1.0, 1.0,], // cyan  
                    [1.0, 1.0, 1.0, 1.0,] ]; // white
```

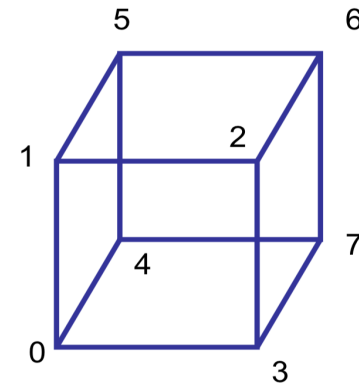




Definiamo un Cubo (3/5)

Definiamo le facce:

```
function colorCube()  
{  
    quad(1, 0, 3, 2);  
    quad(2, 3, 7, 6);  
    quad(3, 0, 4, 7);  
    quad(6, 5, 1, 2);  
    quad(4, 5, 6, 7);  
    quad(5, 4, 0, 1);  
}
```





Definiamo un Cubo (4/5)

Definiamo ogni faccia con un colore (e precisamente con il colore del primo vertice che definisce la faccia):

```
function quad(a, b, c, d) {  
    pointsArray.push(vertices[a]);  
    colorsArray.push(vertexColors[a]);  
    pointsArray.push(vertices[b]);  
    colorsArray.push(vertexColors[a]);  
    pointsArray.push(vertices[c]);  
    colorsArray.push(vertexColors[a]);  
    pointsArray.push(vertices[a]);  
    colorsArray.push(vertexColors[a]);  
    pointsArray.push(vertices[c]);  
    colorsArray.push(vertexColors[a]);  
    pointsArray.push(vertices[d]);  
    colorsArray.push(vertexColors[a]);  
}
```




Definiamo un Cubo (5/5)

In questo modo stiamo definendo due array rispettivamente di vertici e colori da copiare in due VBO:

```
colorCube();  
pointsArray=m4.flatten(pointsArray);  
colorsArray=m4.flatten(colorsArray);
```

dove `m4.flatten` è necessario per ottenere un array tipizzato unidimensionale, come richiede WebGL:

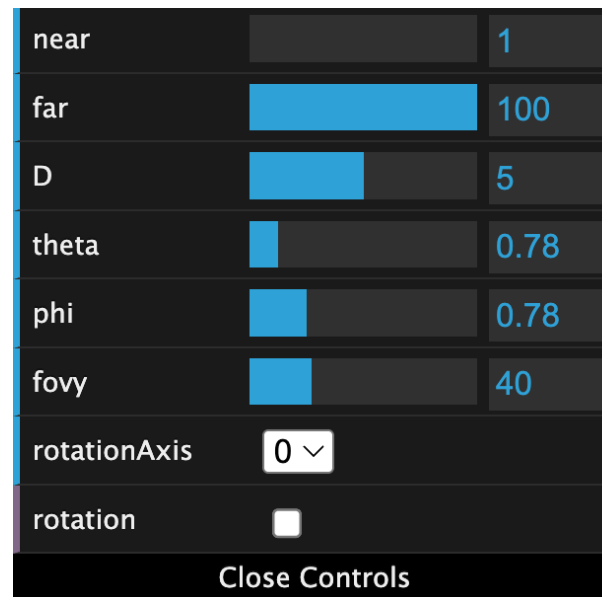


Utilizzo della libreria dat.gui.js

Il codice di esempio definisce la GUI di figura che permette di modificare i parametri di vista e di proiezione, oltre all'asse su cui ruotare il cubo.

Si definisce la seguente struttura:

```
var controls = {  
  near : 1,  
  far : 100,  
  D : 5.0,  
  theta : 0.78,  
  phi : 0.78,  
  fovy : 40.0, //angle (in degrees)  
  rotationAxis: 0,  
  rotation : false  
}
```

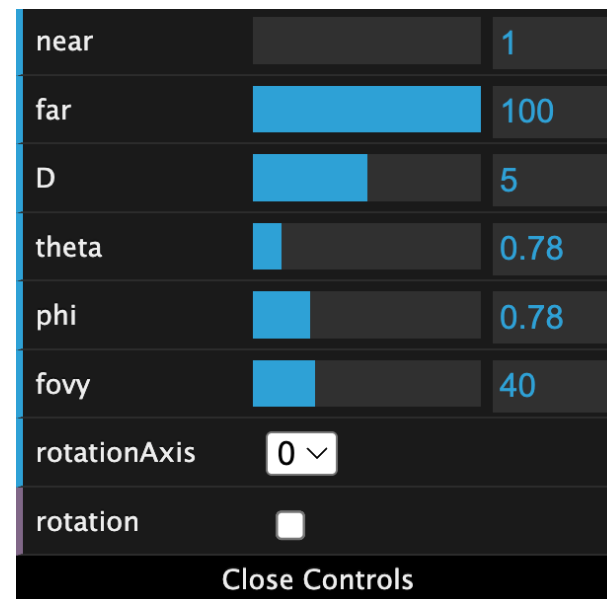




Utilizzo della libreria dat.gui.js

Si definisce poi la seguente funzione:

```
function define_gui( ){  
  
    var dr = 5.0 * Math.PI/180.0  
    var gui = new dat.GUI( );  
  
    gui.add(controls,"near").min(1).max(10).step(1);  
    gui.add(controls,"far").min(1).max(100).step(1);  
    gui.add(controls,"D").min(0).max(10).step(1);  
    gui.add(controls,"theta").min(0).max(6.28).step(dr);  
    gui.add(controls,"phi").min(0).max(3.14).step(dr);  
    gui.add(controls,"fovy").min(10).max(120).step(5);  
    gui.add(controls,"rotationAxis", [0, 1, 2]);  
    gui.add(controls,"rotation");  
}
```





Utilizzo della libreria dat.gui.js

Nel main si chiama la funzione `define_gui` precedentemente definita:

```
define_gui();
```

Da adesso in poi si possono utilizzare i Valori definiti in `controls` eventualmente modificati tramite la GUI; per esempio per la camera:

```
eye = [controls.D*Math.sin(controls.phi)*Math.cos(controls.theta),  
        controls.D*Math.sin(controls.phi)*Math.sin(controls.theta),  
        controls.D*Math.cos(controls.phi)];
```

per la proiezione prospettica:

```
var pMatrix = m4.perspective(degToRad(controls.fovy), aspect,  
                             controls.near, controls.far);
```

per la rotazione:

```
If ( controls.rotation ) {  
  
}
```



Libreria dat.gui.js

Il comando

```
gui.add(object, property, [min], [max], [step])
```

nella libreria **dat.gui.js** è estremamente versatile perché crea automaticamente il controllo più adatto in base al tipo di dato della proprietà specificata.

Ecco le principali possibilità e i controlli che si possono generare:

1. Controlli Numerici (Slider e Input)

Se property è un numero, dat.gui crea un campo di inserimento. Se aggiungi i parametri di minimo e massimo, diventa uno slider.

- **Campo numerico semplice:** `gui.add(obj, 'prop');`
- **Slider con range:** `gui.add(obj, 'prop', 0, 100);`
- **Slider con step (incremento):** `gui.add(obj, 'prop', 0, 100, 5);`
per muoversi, ad esempio, solo di 5 in 5.



Libreria dat.gui.js

2. Menù a Discesa (Dropdown)

Puoi creare un menù a tendina passando un array o un oggetto come terzo parametro:

- **Con Array:** `gui.add(obj, 'prop', ['Opzione 1', 'Opzione 2']);`
- **Con Oggetto (Etichetta: Valore):** `gui.add(obj, 'prop', { Basso: 0, Medio: 50, Alto: 100 });`

3. Checkbox (Booleani)

Se la property di object è un valore booleano (true o false), dat.gui genererà automaticamente una casella di controllo.

- Esempio: `gui.add(obj, 'visibile');`

4. Campi di Testo (Stringhe)

Per le property di tipo stringa, viene creato un campo di testo dove l'utente può digitare liberamente.

- Esempio: `gui.add(obj, 'nomeUtente');`



Libreria dat.gui.js

5. Pulsanti (Funzioni)

Se la property puntata è una funzione, dat.gui crea un pulsante che, se cliccato, esegue quella funzione.

- Esempio: `gui.add(obj, 'salvaDati');` (dove `salvaDati` è una funzione definita in `obj`).

6. Metodi di Concatenazione (Chaining)

Ogni comando `gui.add` restituisce un controller su cui puoi concatenare metodi per rifinire il comportamento:

- `.name("Nuovo Nome")`: Cambia l'etichetta visualizzata nella GUI.
- `.onChange(callback)`: Esegue una funzione ogni volta che il valore cambia.
- `.onFinishChange(callback)`: Esegue una funzione solo quando l'utente smette di interagire (es. rilascia lo slider).
- `.listen()`: Forza la GUI ad aggiornarsi se il valore della variabile cambia esternamente via codice.



Libreria glm_utils.js

La libreria **glm_utils.js** è presente nella cartella:

HTML5_webgl_3/resources

Tra le varie function vediamo in particolare:

glmReadOBJ: carica un file in formato OBJ Wavefront e ritorna in una struttura **subd_mesh** tutte le informazioni utili per il disegno di una mesh (vertici, facce, normali, coordinate texture, ecc.);

Unitize: scala e trasla una mesh per portarla in una sfera unitaria centrata nell'origine.

Si veda il codice di esempio:

view_obj_file_webgl.html + .js



Libreria glm_utils.js

Il codice di esempio:

`view_obj_file_webgl.html + .js`

per caricare un file .obj utilizza la function

`LoadMesh()`

definita nel file `load2_mesh.js` che utilizza le strutture definite nella libreria `mesh_utils.js` e le function definite nella `glm_utils.js`



Libreria mesh_utils.js

La libreria `mesh_utils.js` è presente nella cartella:

`HTML5_webgl_3/resources`

Oltre a definire le strutture dati utili per mesh, fra le varie function citiamo in particolare la:

LoadSubdMesh: partendo da una definizione di mesh in termini di vertici V e facce FV (Vertici di ogni Faccia), determina tutte le informazioni utili per la gestione di **unstructured mesh** (progetto **subdivision surfaces**).

(la riprenderemo anche più avanti)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it